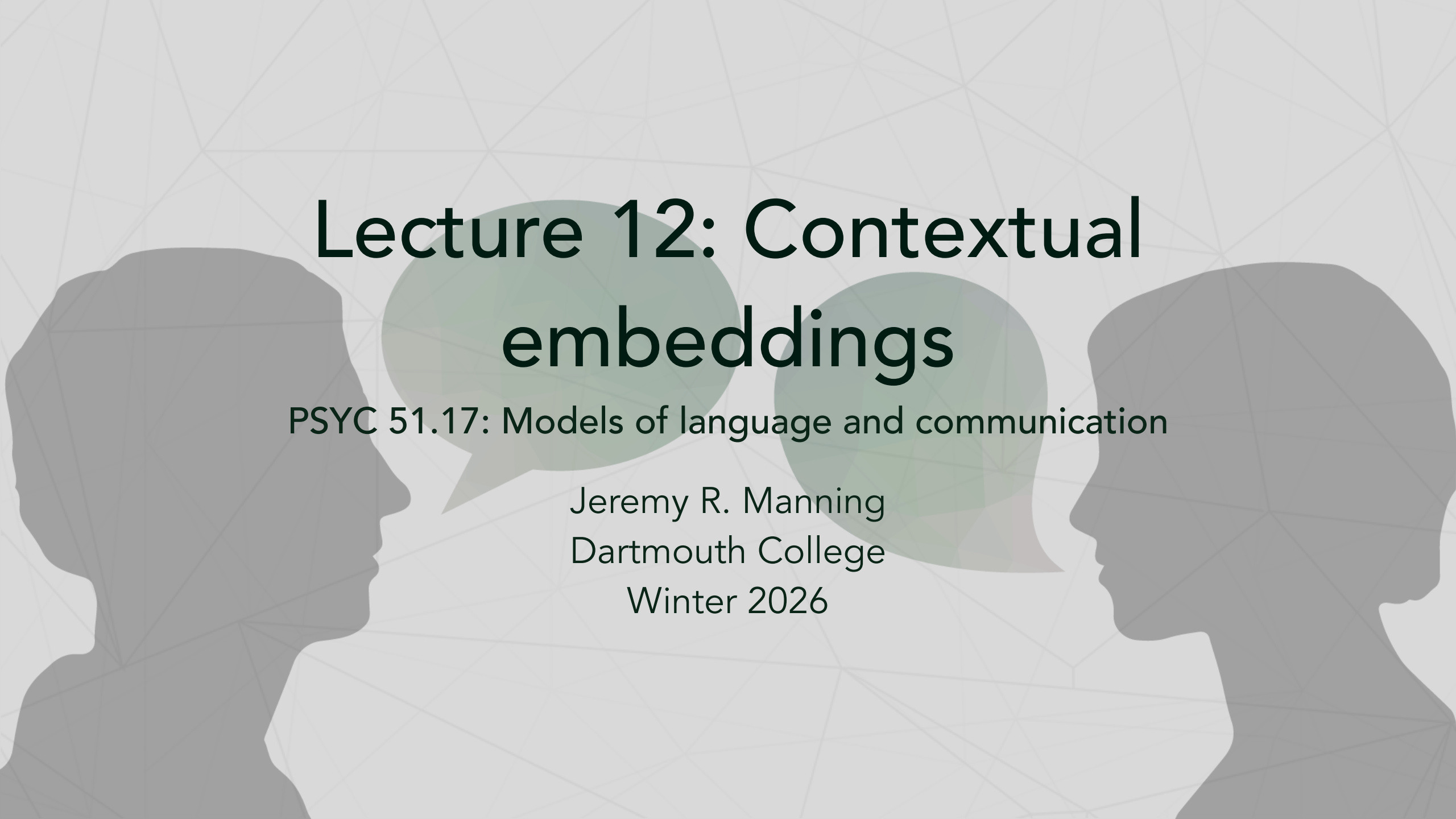




Lecture 12: Contextual embeddings

PSYC 51.17: Models of language and communication

Jeremy R. Manning
Dartmouth College
Winter 2026

Learning objectives

By the end of this lecture, you will

1. Understand the polysemy problem and why static embeddings struggle
2. Explain how ELMo uses bidirectional LSTMs for context
3. Apply Universal Sentence Encoder for semantic similarity
4. Understand BERT's masked language modeling approach
5. Compare contextual embedding methods and their trade-offs

The contextual revolution

Contextual embeddings changed NLP: same word, different vectors depending on context!

The polysemy problem

Static embeddings: ONE vector per word

Word2Vec, GloVe, and FastText assign the **same** vector to every occurrence of a word.

The word "bank"

1. "I deposited money at the **bank**" (financial institution)
2. "We sat by the river **bank**" (riverside)
3. "The plane will **bank** left" (tilt/turn)

Static embeddings: All three get the SAME vector!

Contextual embeddings: Each gets a DIFFERENT vector based on context.

Static embeddings (Word2Vec, GloVe, FastText)

Same vector every time

```
1 model = Word2Vec( ... )  
2 vec1 = model['bank']  
3 vec2 = model['bank']  
4 assert vec1 == vec2 # Always True!
```

Characteristics

- **One vector per word type** — lookup table
- **Context-independent** — ignores surrounding words
- **Fast** — dictionary lookup, no computation

Contextual embeddings (ELMo, BERT)

Different vector per occurrence

```
1 sent1 = "river bank"
2 sent2 = "money bank"
3 vec1 = get_embedding(model, sent1, 'bank')
4 vec2 = get_embedding(model, sent2, 'bank')
5 assert vec1  $\neq$  vec2 # Always True!
```

Characteristics

- **Different vector per occurrence** — context-dependent
- **Bidirectional context** — sees words before AND after
- **Slower** — requires forward pass through neural network

Language models as feature extractors

Key insight

Train a language model, use its internal states as embeddings!

Modern language models are predictors

Fundamental goal: predict the next word given previous words: $P(w_t | w_1, w_2, \dots, w_{t-1})$

Concrete example

Input: "The cat sat on the ____"

Model predicts: "mat" (0.25), "floor" (0.18), "couch" (0.12), "dog" (0.001)

To predict well, the model learns: "sat on the" suggests a surface!

Why this works

Hidden states capture contextual understanding. We extract these rich hidden states as embeddings!

ELMo: Embeddings from Language Models

Further reading

Peters, Neumann, Iyyer, Gardner, Clark, Lee, Zettlemoyer (2018, *NAACL*). Deep contextualized word representations.

The first widely-adopted contextual embedding (2018)

- Train deep **bidirectional** language model
- Use all layer activations
- Weighted combination per task
- Character-based input (handles OOV!)

ELMo architecture

Bidirectional LSTM

- **Forward LM:** $P(w_t | w_1 \dots w_{t-1})$
- **Backward LM:** $P(w_t | w_{t+1} \dots w_n)$

Each word gets info from BOTH directions!

Layer weighting

For sentiment analysis, ELMo might learn:

- Layer 0 (characters): weight = 0.1
- Layer 1 (syntax): weight = 0.3
- Layer 2 (semantics): weight = 0.6

Higher layers matter more for meaning!

Impact on NLP

Improved state-of-the-art on many NLP tasks: question answering, sentiment analysis, named entity recognition, coreference resolution, semantic role labeling, and more!

ELMo in Python

```
1  from allennlp.modules.elmo import Elmo, batch_to_ids
2
3  # Initialize ELMo (need to download these files first)
4  options_file = "elmo_options.json"
5  weight_file = "elmo_weights.hdf5"
6  elmo = Elmo(options_file, weight_file, 2, dropout=0)
7
8  # prepare inputs and get embeddings
9  sentences = [
10     ['I', 'deposited', 'money', 'at', 'the', 'bank'],
11     ['We', 'sat', 'by', 'the', 'river', 'bank']
12 ]
13 character_ids = batch_to_ids(sentences)
14 embeddings = elmo(character_ids)
15
16 # Different vectors for "bank"!
17 bank1 = embeddings['elmo_representations'][0][0, 5, :] # first sentence
18 bank2 = embeddings['elmo_representations'][0][1, 5, :] # second sentence
```

Universal Sentence Encoder (USE)

Further reading

Cer et al. (2018, *EMNLP*) Universal Sentence Encoder.

Sentence-level embeddings

- Produces 512-dimensional vector for any sentence
- Two variants: Transformer (higher accuracy) and Deep Averaging Network (DAN; faster)
- Optimized for semantic similarity tasks
- Performs really well in practice (still widely used today!)

Use cases

Semantic search, question answering, text classification, clustering, duplicate detection.

Universal Sentence Encoder in Python

```
1 import tensorflow_hub as hub
2 import numpy as np
3 from sklearn.metrics.pairwise import cosine_similarity
4
5 # Load model
6 embed = hub.load("https://tfhub.dev/google/universal-sentence-encoder/4")
7
8 sentences = [
9     "The cat sat on the mat.",
10    "A feline rested on the rug.", # Paraphrase!
11    "The dog ran in the park.",
12    "I love machine learning."
13 ]
14
15 embeddings = embed(sentences) # shape: [4, 512]
16
17 # Compute similarity - sentences 1 and 2 should be very similar
18 sim_matrix = cosine_similarity(embeddings)
```

BERT: Bidirectional Encoder Representations

Further reading

Viswani et al. (2017, *NIPS*) Attention is All You Need.

Devlin, Chang, Lee, Toutanova (2019, *NAACL*) BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding.

Transformer architecture

We will cover Transformers in detail later. For now: it's a very powerful architecture for sequence modeling. The core idea is **self-attention**: each word "attends to" (i.e., is affected by) all other words in the sequence, capturing rich dependencies.

Quickly surpassed ELMo (2018)

- **Bidirectional** context (like ELMo, but captures much deeper patterns due to Transformer)
- **Massive** pre-training (3.3B word corpus)
- Two pre-training tasks: Masked Language Modeling (MLM) & Next Sentence Prediction (NSP)
- Fine-tunable for many downstream tasks

Impact

SOTA on 11 NLP tasks. Sparked the "BERT-era" with 1000+ variants (RoBERTa, ALBERT, DistilBERT, ...). We've already used some of these (e.g., DistilBERT for sentiment analysis)!

Masked Language Modeling (MLM)

A problem with traditional LM

- Left-to-right: only sees **previous** words
- Right-to-left: only sees **next** words
- Ideally we would see both directions simultaneously...
- ...but we can't just show the answer during training!

Solution: Mask some words, predict them

Original: "The cat sat on the mat"

Masked: "The [MASK] sat on the mat"

Task: Predict "cat" using context from BOTH sides!

MLM training procedure

Worked example

Step 1: Randomly select 15% of tokens → "cat" selected

Step 2: Apply masking strategy (80/10/10 rule):

- 80%: "The **[MASK]** sat on the mat"
- 10%: "The **cat** sat on the mat" (unchanged)
- 10%: "The **answer** sat on the mat" (random word)

Step 3: Model predicts "cat" using full bidirectional context

Why 80/10/10?

Prevents model from only learning to predict [MASK] tokens. Forces it to make good predictions for any token.

Next Sentence Prediction (NSP)

Second pre-training task

Given two sentences A and B, predict if B follows A in the text.

Positive example (IsNext)

A: "The cat sat on the mat."

B: "It was sleeping peacefully."

Label: IsNext

Negative example (NotNext)

A: "The cat sat on the mat."

B: "Machine learning is fascinating."

Label: NotNext

Note

Later research (RoBERTa) found NSP less important than MLM for most tasks.

BERT input representation

Three embeddings summed for each token

Input: "[CLS] I love NLP [SEP] It is fun [SEP]"

Token	Token Emb	Segment	Position
[CLS]	E_CLS	A	0
I	E_I	A	1
love	E_love	A	2
NLP	E_NLP	A	3
[SEP]	E_SEP	A	4
It	E_It	B	5
is	E_is	B	6
fun	E_fun	B	7

Final embedding = Token + Segment + Position

BERT in Python

```
1  from transformers import BertTokenizer, BertModel
2  import torch
3
4  tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
5  model = BertModel.from_pretrained('bert-base-uncased')
6
7  sent1, sent2 = "I deposited money at the bank", "We sat by the river bank"
8
9  # Tokenize and get embeddings
10 tokens1 = tokenizer(sent1, return_tensors='pt')
11 tokens2 = tokenizer(sent2, return_tensors='pt')
12
13 with torch.no_grad():
14     output1 = model(**tokens1)
15     output2 = model(**tokens2)
16
17 # Extract "bank" embedding - different vectors!
18 bank1 = output1.last_hidden_state[0, 6, :] # 768-dim
19 bank2 = output2.last_hidden_state[0, 6, :] # 768-dim
```

Fine-tuning BERT

What is fine-tuning?

Fine-tuning: starting with pre-trained model weights, then updating them on a specific downstream task (e.g., sentiment analysis, question answering).

Why fine-tune?

- Leverages knowledge from massive pre-training
- Adapts model to specific task/domain
- Much faster than training from scratch

Fine-tuning BERT

Feature extraction (freeze BERT)— fast, less data needed

```
1 for param in bert_model.parameters():  
2     param.requires_grad = False  
3 classifier = nn.Linear(768, num_classes)  
4 # Train only classifier
```

Fine-tuning (update BERT)— performant, needs more data

```
1 bert_model = BertModel.from_pretrained('bert-base-uncased')  
2 classifier = nn.Linear(768, num_classes)  
3 optimizer = Adam([ ... ], lr=2e-5) # Small learning rate!  
4 # Train everything end-to-end
```

Best practice

Fine-tune with small learning rate (1e-5 to 5e-5) for few epochs (2-4).

Real-world applications

Google search

Query: "can you get medicine for someone pharmacy"

BERT understands: picking up a prescription FOR someone else (not just keyword matching!)

Question answering

Context: "The Eiffel Tower was built in 1889 by Gustave Eiffel."

Q: "When was the Eiffel Tower built?"

A: "1889" ← BERT extracts this span

Sentiment analysis

"Not bad at all!" → Positive (understands negation!)

Named entity recognition

"Apple is looking at buying U.K. startup for \$1 billion" → Recognizes "Apple" as ORG, "U.K." as LOC, "\$1 billion" as MONEY.

Discussion: do contextual embeddings reflect "understanding"?

In some ways...

- Capture complex semantic relationships
- Generalize to new contexts
- Handle compositional meaning (e.g., idioms, metaphors, sarcasm)
- Human-level performance on many benchmarks

...but probably not true understanding

- Brittle to adversarial examples (small changes can fool them)
- Lack of world knowledge and reasoning: grammatical but nonsensical sentences can be rated as highly probable
- Struggle with out-of-distribution inputs that humans handle easily (e.g., novel metaphors, jokes)

...on the other hand

Notice that with these models we are far beyond pre-defined symbolic rules and simple co-occurrence statistics. They capture a lot of nuance in language use that was previously out of reach. Arguing "where models fall short of true understanding" is (increasingly) becoming a moving target, and increasingly subtle.

...but on the OTHER other hand

Lack of grounding in actual experience means that these models cannot distinguish between reasonable vs. unreasonable statements that happen to be linguistically well-formed. Whatever "understanding" these models might be said to have is fundamentally different from human understanding: it is based purely on patterns in text, without any connection to real-world referents or experiences.

Summary

1. **Contextual vs. static:** Different vectors per occurrence solves polysemy
2. **ELMo (2018):** BiLSTM language models, character-based, task-specific weighting
3. **USE (2018):** Sentence-level embeddings optimized for similarity
4. **BERT (2018):** Masked language modeling, deep bidirectional transformers
5. **Impact:** Established modern transfer learning paradigm in NLP

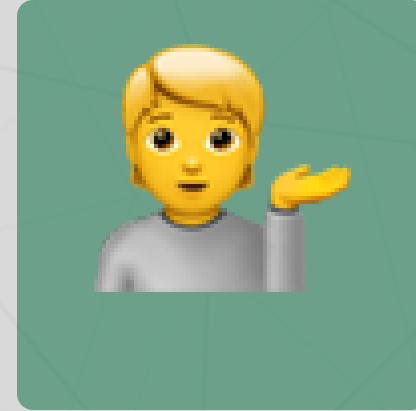
Questions?



Email me



Join our Discord



Come to office hours

Up next (tomorrow's X-hour)

Dimensionality reduction techniques (PCA, t-SNE, UMAP) for visualizing embeddings!